

Running in Production with Docker and Fly.io

Contents

1. Purpose and scope	3
2. The two runtime requirements	4
3. Part 1 — Preventing the missing <code>AGE_KEY</code> error	5
3.1. Injecting the key at container start	5
3.2. Using an untracked env file for local development	5
3.3. Configuring the key as a Fly.io secret	6
4. Part 2 — Preventing the database <code>localhost</code> error	7
5. Part 3 — Docker Compose	9
5.1. The <code>compose.yaml</code> file	9
5.2. Why each field is there	10
5.3. Running it	11
5.4. Verifying the setup	12
6. Part 4 — Fly.io deployment	15
6.1. One-time setup	15
6.2. What goes in <code>fly.toml</code>	15
6.3. What must not go in <code>fly.toml</code>	16
6.4. Verifying the deployment	16
6.5. Rotating <code>AGE_KEY</code>	17
7. Part 5 — Keeping production values out of local files	18
8. Part 6 — Troubleshooting	19
8.1. <code>no age key available</code>	19

8.2. failed to decrypt secrets.enc.yaml: no identity matched any of the recipients	19
8.3. connection refused or dial tcp ... connect: connection refused on the database	19
8.4. password authentication failed for user "... "	20
8.5. Port 8080 is in use.....	20
8.6. The Compose healthcheck never reports healthy	20
8.7. pull access denied for mywebapp on docker compose up.....	21
8.8. The app starts but /health returns degraded	21
8.9. docker compose up cannot find host.docker.internal	21
9. Summary	22
10. Files referenced by this document	23

patrick toca v0.8.3, October 7th, 2026

Creation Date: October 7th, 2026

Author: patrick toca

Email: pmjtoca@gmail.com

1. Purpose and scope

This document describes how to run the MyWebApp container in production and in local environments, focusing on the runtime configuration the image needs and the failures that occur when it is missing.

The container image is self-contained. Every failure observed so far has been caused by configuration that must be supplied at **runtime**, not at build time. Two failures dominate:

1. A missing `AGE_KEY`, which the application requires to decrypt its secrets file at boot. The process exits immediately when it is absent.
2. A `DB_HOST` value of `localhost`, which inside a container refers to the container itself, not the host running PostgreSQL.

Both are addressed in this document. The Compose setup described here has been verified end-to-end: the image builds locally, the app connects to PostgreSQL, port 8080 is published, and `/health` returns `{"status":"ok"}`.

By the end of this document you will be able to:

1. Run the container locally against a PostgreSQL instance on the host.
2. Use an untracked env file to keep runtime secrets out of the image and out of git.
3. Bring the same setup up with `docker compose`.
4. Deploy the same image to Fly.io with `AGE_KEY` stored as a Fly Secret.
5. Diagnose the two common runtime failures from the container logs.

2. The two runtime requirements

The application will not start without both of these:

1. **AGE_KEY** — the age private key used by `internal/credentials` to decrypt `secrets.enc.yaml`. Without it, the process exits with a clear error before any service is constructed.
2. **A reachable PostgreSQL instance** — the connection is established at startup. The host must be set correctly for the container's network namespace.

Everything else — port, log format, cookie flags, CSP policy — has a safe default and can be overridden if needed.

3. Part 1 — Preventing the missing `AGE_KEY` error

The application fails fast when `AGE_KEY` is absent. This is intentional: a missing decryption key must never be silently ignored. A server that starts without it is a server that will fail later, under load, in a harder-to-diagnose way.

3.1. Injecting the key at container start

Pass the key directly when starting the container. Read it from your local age key file, stripping comment lines:

```
docker run --rm \
  -e AGE_KEY="$(grep -v '^#' ~/.config/sops/age/keys.txt)" \
  ...
mywebapp:debug
```



Passing secrets on the command line exposes them to shell history and to any user who can run `ps`. This form is acceptable only for a quick local test. Use an env file or a platform secret for anything persistent.

3.2. Using an untracked env file for local development

Create a local environment file containing the required runtime values:

```
AGE_KEY=AGE-SECRET-KEY-...
DB_HOST=host.docker.internal
DB_PORT=5432
DB_USER=pmjtoca
DB_NAME=MyWebApp
DB_SSLMODE=disable
PORT=8080
```

Then start the container with that file:

```
docker run --rm \
  --env-file .env.docker.local \
  --add-host host.docker.internal:host-gateway \
  -p 8080:8080 \
  mywebapp:debug
```

`PORT=8080` is set explicitly in the file. The image's default port is already 8080, but setting it in the env file makes the Compose port mapping and the application's listen port agree without relying on the default.

Add the env file to `.gitignore` so it is never committed:

```
.env.docker.local
.env.docker.local.*
```



Never put `AGE_KEY` in the Dockerfile, in image layers, in Git, or in a committed `.env` file. Any of those locations turns a runtime secret into a permanent, widely distributed artifact.

3.3. Configuring the key as a Fly.io secret

On Fly.io, store the key as a platform secret rather than an environment file:

```
fly secrets set AGE_KEY="$(grep -v '^#' ~/.config/sops/age/keys.txt)" --app mywebapp
```

Fly injects the secret into the running machine at start time, so it never appears in the image or in a repository.

4. Part 2 — Preventing the database `localhost` error

Inside a container, `localhost` refers to the container's own network namespace. PostgreSQL is almost never there. Set `DB_HOST` according to where the database actually runs:

Where PostgreSQL runs	Value for <code>DB_HOST</code>
On the host machine	<code>host.docker.internal</code>
In Docker Compose	The database service name, for example <code>db</code>
On Fly.io	The private or managed database hostname
Anywhere else	Its reachable DNS name or IP address

Your local `docker run` command should therefore include:

```
-e DB_HOST=host.docker.internal
```

On Linux, `host.docker.internal` does not resolve by default. Add the mapping explicitly:

```
--add-host host.docker.internal:host-gateway
```

On Docker Desktop for macOS and Windows, the mapping is provided automatically; the `--add-host` flag is harmless but unnecessary there.

In addition to setting `DB_HOST`, ensure that PostgreSQL itself accepts connections from Docker:

- Its `listen_addresses` setting must include the interface the container reaches it on. For a PostgreSQL listening only on `localhost`, set it to `'*'` or to the Docker bridge address.
- Its `pg_hba.conf` must contain a rule permitting the Docker network or the container's source address, with `md5`, `scram-sha-256`, or `trust` as appropriate.
- Any host firewall must allow traffic from the Docker bridge network to the PostgreSQL port. On a host with `ufw`, the Docker bridge

interface (`docker0`) is usually already permitted; on a host with `firewalld`, an explicit rule may be required.

5. Part 3 — Docker Compose

For local development, replace ad-hoc `docker run` invocations with a Compose file. This makes the runtime configuration explicit and reproducible, and it is what the repository ships with.

5.1. The `compose.yaml` file

```
services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
    image: mywebapp:local
    pull_policy: never          # never try to pull; build locally
    ports:
      - "8080:8080"
    env_file:
      - .env.docker.local
    extra_hosts:
      - "host.docker.internal:host-gateway"
    networks:
      - mywebapp-net
    healthcheck:
      test: ["CMD", "wget", "-q0-", "http://localhost:8080/health"]
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 10s
      restart: unless-stopped

networks:
  mywebapp-net:
```

```
driver: bridge
```

5.2. Why each field is there

`build.context` and `dockerfile`

Build the image from the repository root using the project's `Dockerfile`. There is no registry involved.

`image: mywebapp:local` and `pull_policy: never`

Tag the locally built image and instruct Compose not to attempt a registry pull. This is what "prevents registry pulls" in the commit message refers to. Without `pull_policy: never`, a `docker compose up` can go looking for `mywebapp:local` on Docker Hub and fail if a network blip occurs.

`ports`

Publish the application's listen port on the host. The left side is the host port; the right side is the container port. Both are 8080 because `.env.docker.local` sets `PORT=8080`.

`env_file`

Read `.env.docker.local` and pass each key as an environment variable into the container. This is where `AGE_KEY`, `DB_HOST`, and the rest of the runtime configuration live.

`extra_hosts`

Map `host.docker.internal` to the host gateway inside the container. On Linux, Docker does not provide this name by default. This entry is what makes the `DB_HOST=host.docker.internal` value in `.env.docker.local` resolve to the host running PostgreSQL.

`networks`

Attach the container to a named bridge network. A named network is used rather than the default `bridge` because the default network has a fixed set of characteristics — no automatic service discovery by name, a different gateway address — that have caused problems in the past. The named network behaves predictably.

`healthcheck.test`

A `wget` against `/health`. The endpoint returns `{"status":"ok"}` only when the database ping succeeds, so the healthcheck is a real liveness

signal, not just "the HTTP server is listening." `wget -q0-` is a GET request; `--spider` was the form used in earlier iterations and was not compatible with every `wget` build. The GET form is what "GET-compatible healthcheck" in the commit message refers to.

`healthcheck.interval`, `timeout`, `retries`, `start_period`

Reasonable defaults. `start_period: 10s` gives the application time to load secrets and open the database pool before the first healthcheck runs.

`restart: unless-stopped`

Restart the container if it exits, except when it was explicitly stopped by the user. This is the standard choice for a long-running service in a local Compose setup.

5.3. Running it

```
docker compose up --build
```

The first run builds the image. Subsequent runs reuse it. To force a rebuild after a code change:

```
docker compose up --build --force-recreate
```

To run in the background:

```
docker compose up -d
```

To stop and remove the container:

```
docker compose down
```

5.4. Verifying the setup

The commit that fixed the Compose setup verified four things. The same four checks apply to any new environment.

5.4.1. 1. The image builds locally

```
docker compose build
```

Expected: a successful build, ending with a **Successfully tagged mywebapp:local** line. If the build fails, the error is usually a missing file referenced by the **Dockerfile** or a Go compilation error; the output names the file and line.

5.4.2. 2. The app connects to PostgreSQL

```
docker compose logs app | grep -i "database\|postgres\|pool"
```

Expected: a log line indicating the pool was configured. If the app cannot reach PostgreSQL, the log contains a connection error naming the host and port it tried.

If the app is running but the log shows a retry loop, confirm that:

- **DB_HOST** in **.env.docker.local** is **host.docker.internal**, not **localhost**.
- PostgreSQL is listening on an interface reachable from the Docker bridge, not only on the host's loopback.
- **pg_hba.conf** permits the container's source address.

5.4.3. 3. Port 8080 is published

```
docker compose ps
```

Expected: a line showing `0.0.0.0:8080->8080/tcp` in the `PORTS` column. If the column shows only `8080/tcp`, the `ports:` mapping in `compose.yaml` was not applied — check that the indentation is correct and that the service is the one you edited.

5.4.4. 4. `/health` returns `{"status":"ok"}`

```
curl -s http://localhost:8080/health
```

Expected:

```
{
  "status": "ok",
  "time": 1758567833,
  "pool": {
    "total": 5,
    "idle": 5,
    "acquired": 0,
    "max": 25,
    "constructing": 0,
    "acquire_count": 12,
    "empty_acquire": 0,
    "canceled_acquire": 0,
    "acquire_duration_ms": 3
  }
}
```

The `pool` block is present because `/health` pings the database. A `status` of `ok` means the ping succeeded. A `503` or a `status` of `degraded` means the database is not reachable; the `pool` block will show `total: 0` or a non-zero `canceled_acquire`.

5.4.5. 5. The Compose healthcheck reports healthy

```
docker compose ps
```

Expected: a **STATUS** of **Up ... (healthy)** after the **start_period** elapses. A status of **(unhealthy)** after the retry count means the healthcheck is failing. Check **docker compose logs app** for the underlying error, and confirm that **wget** is available inside the image — the healthcheck depends on it.

6. Part 4 — Fly.io deployment

Fly.io runs the same image. The only difference is where the runtime configuration comes from: an env file locally, Fly Secrets in production.

6.1. One-time setup

```
fly auth login
fly launch --no-deploy      # generates/updates fly.toml, if needed
fly secrets set AGE_KEY="$(grep -v '^#' ~/.config/sops/age/keys.txt)"
fly deploy
```

`AGE_KEY` is the only Fly Secret that needs to be set. Every other secret — `JWT_SECRET`, `CSRF_SECRET`, `DB_PASSWORD`, the R2 credentials — lives in `secrets.enc.yaml`, which is decrypted at boot by `internal/credentials`. Storing them again as Fly Secrets would create a second source of truth and defeat the design. See [the age/SOPS/Fly how-to](#) for the full model.

6.2. What goes in `fly.toml`

Non-secret configuration. The `[env]` block is visible in the Fly dashboard and in the file itself, so it holds only values that are not sensitive:

```
[env]
  PORT = "8080"
  GIN_MODE = "release"
  LOG_FORMAT = "json"

[http_service]
  internal_port = 8080
  force_https = true
  auto_stop_machines = "stop"
  auto_start_machines = true
  min_machines_running = 0
```

```
[[http_service.checks]]
  interval = "30s"
  timeout = "5s"
  grace_period = "10s"
  method = "GET"
  path = "/health"
```

`internal_port` matches the `PORT` the application listens on. The `healthcheck` block is the Fly equivalent of the Compose healthcheck; it hits `/health` with a GET, which is the same GET-compatible form the Compose file uses.

6.3. What must not go in `fly.toml`

- `AGE_KEY` — it is a secret and belongs in `fly secrets`.
- `JWT_SECRET`, `CSRF_SECRET` — same.
- `DB_PASSWORD` — same.
- Any R2 credential — same.

If a value in `[env]` would be redacted in a log line, it belongs in `fly secrets`, not in `[env]`.

6.4. Verifying the deployment

```
fly status
fly logs
curl -s https://mywebapp.fly.dev/health | jq .
curl -s https://mywebapp.fly.dev/version | jq .
```

Expected: `status` is `ok`, the log contains a `credentials loaded` line with a non-zero count, and `/version` reports the tag and commit that were deployed.

6.5. Rotating AGE_KEY

See the "Rotating the age key" section of [the age/SOPS/Fly how-to](#). The short version:

```
# 1. Generate a new key, re-encrypt secrets.enc.yaml to it, keep the old
#    recipient as a fallback.
# 2. Set the new key on Fly.
fly secrets set AGE_KEY="$(cat ~/.config/sops/age/keys-new.txt)"

# 3. Deploy and confirm healthy.
fly deploy
fly logs

# 4. Remove the old recipient from secrets.enc.yaml, commit, deploy.
```

The re-encryption must happen before the old key is discarded. There is no way to decrypt a file with a key it was not encrypted to.

7. Part 5 — Keeping production values out of local files

Local env files are a convenience for development only. For production:

- Keep every secret in Fly secrets, or your platform's equivalent secret store.
- Keep non-secret configuration in the deployment configuration, not in a committed env file.
- Keep the application's fail-fast behavior. The process should continue to exit immediately when the age key or the database configuration is missing.

The `.gitignore` entries that support this:

```
.env
.env.*
!.env.example
.env.docker.local
.env.docker.local.*
secrets.yaml
secrets.yml
secrets.env
```

`secrets.enc.yaml` is deliberately **not** ignored. It is the ciphertext, it is safe to commit, and `git diff` on it is meaningful because SOPS encrypts the values but leaves the keys readable.

8. Part 6 — Troubleshooting

8.1. no age key available

The error from `credentials.Load`. Causes, in order of likelihood:

1. `AGE_KEY` is unset. Locally, check `.env.docker.local`. On Fly, check `fly secrets list`.
2. The env file was not read. Confirm the `--env-file` flag or the `env_file:` entry in `compose.yaml`.
3. The key file at `~/.config/sops/age/keys.txt` is missing or has permissions looser than `0600`. `sops` and `age` refuse to read a key file with looser permissions.

8.2. failed to decrypt secrets.enc.yaml: no identity matched any of the recipients

The `AGE_KEY` supplied to the container is not the one the file was encrypted to. Either the wrong key was passed, or the file was re-encrypted after the key was copied. Retrieve the correct key or re-encrypt the file to the current recipient.

8.3. connection refused or dial tcp ... connect: connection refused on the database

The container reached the host but PostgreSQL is not listening on the address it tried. Common causes:

1. `DB_HOST` is `localhost` and the container is talking to itself. Set it to `host.docker.internal` and pass `--add-host host.docker.internal:host-gateway`.
2. PostgreSQL is listening only on `127.0.0.1`. Set `listen_addresses` to `'*'` or to the Docker bridge address.
3. `pg_hba.conf` has no rule for the container's source address.
4. A host firewall is blocking traffic from the Docker bridge.

8.4. password authentication failed for user "..."

The password in `DB_PASSWORD` (loaded from `secrets.enc.yaml`) does not match the password for the role in PostgreSQL. Either update the encrypted file, or change the role's password.

8.5. Port 8080 is in use

Another process holds the port. Find it:

```
sudo ss -ltnp | grep :8080
```

Either stop the other process, or change the host-side port mapping in `compose.yaml`:

```
ports:
  - "8081:8080"
```

The container still listens on 8080; only the host port changes. Update any client that connects to the host port accordingly.

8.6. The Compose healthcheck never reports healthy

Check the log:

```
docker compose logs app
```

If the application is logging normally, the healthcheck itself is failing. Two common causes:

1. `wget` is not present in the image. The healthcheck depends on it. Either add `wget` to the runtime stage of the `Dockerfile`, or change the healthcheck to use a tool the image does have, such as `curl`.

2. The healthcheck runs before the database pool is ready and the retry count is exhausted. Increase `start_period` and `retries` in `compose.yaml`.

8.7. pull access denied for mywebapp on docker compose up

Compose is trying to pull the image from a registry instead of using the locally built one. Confirm that `compose.yaml` has both:

```
image: mywebapp:local
pull_policy: never
```

and that the build succeeds with `docker compose build` before `docker compose up`.

8.8. The app starts but /health returns degraded

The HTTP server is listening but the database ping fails. The `pool` block in the response shows the current pool state; a `total` of `0` or a non-zero `canceled_acquire` points at a connection problem. Check `docker compose logs app` for the underlying error.

8.9. docker compose up cannot find host.docker.internal

On Linux, this name is not provided by default. Confirm that the service has:

```
extra_hosts:
- "host.docker.internal:host-gateway"
```

and that Docker is recent enough to support the `host-gateway` special value (Docker 20.10 or later).

9. Summary

1. Supply `AGE_KEY` at runtime, through an untracked env file locally or a platform secret in production.
2. Set `DB_HOST` to the real location of PostgreSQL, never to `localhost` from inside a container.
3. Use `compose.yaml` for local runs, with `pull_policy: never` to prevent registry pulls, a named bridge network, and `host.docker.internal:host-gateway` for host access.
4. Use a GET-compatible healthcheck against `/health`, which is a real liveness signal because it pings the database.
5. Use Fly secrets for `AGE_KEY` and nothing else; everything else lives in `secrets.enc.yaml`.
6. Preserve fail-fast behavior on missing configuration.

10. Files referenced by this document

Path	Purpose
Dockerfile	The image build definition.
compose.yaml	The local Compose service definition.
.env.docker.local	The untracked local env file. Gitignored. Contains AGE_KEY, DB_HOST, and the other runtime values.
fly.toml	The Fly application manifest. Contains non-secret configuration only.
secrets.enc.yaml	The committed, SOPS-encrypted secrets file. Decrypted at boot by internal/credentials.
internal/credentials/credentials.go	The loader that reads AGE_KEY and exports the decrypted secrets into the environment.
AGE_KEY	The Fly Secret holding the age private key.